Imperial College
London

Lecture 7

Array Subsystems

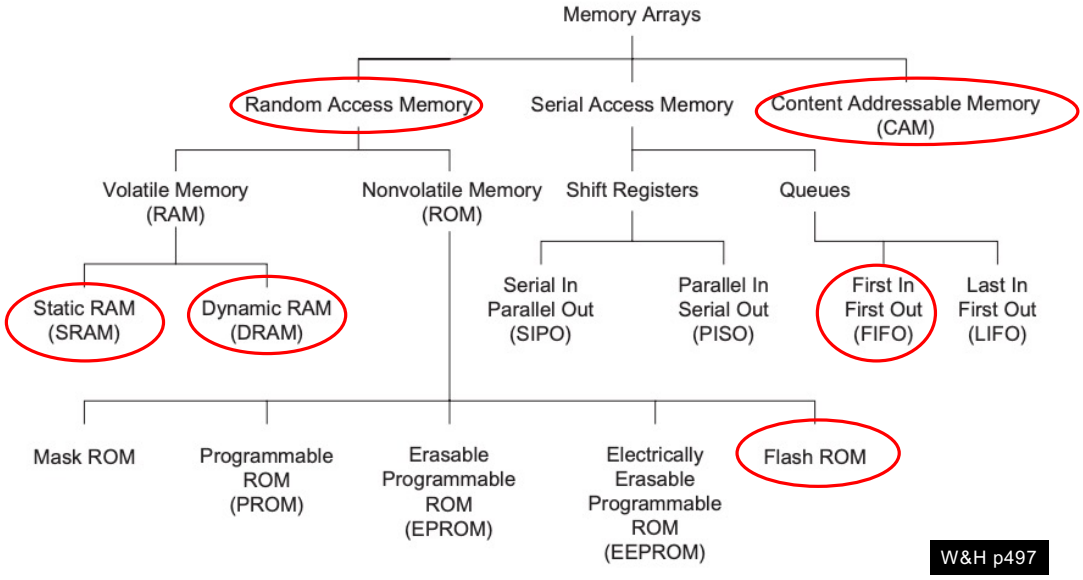
Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: https://elec70142-digital-vlsi-design.github.io/course-website/
E-mail: p.cheung@imperial.ac.uk

In this lecture, we will examine the different way that combinational logic gates are implemented in full-custom VLSI circuits.

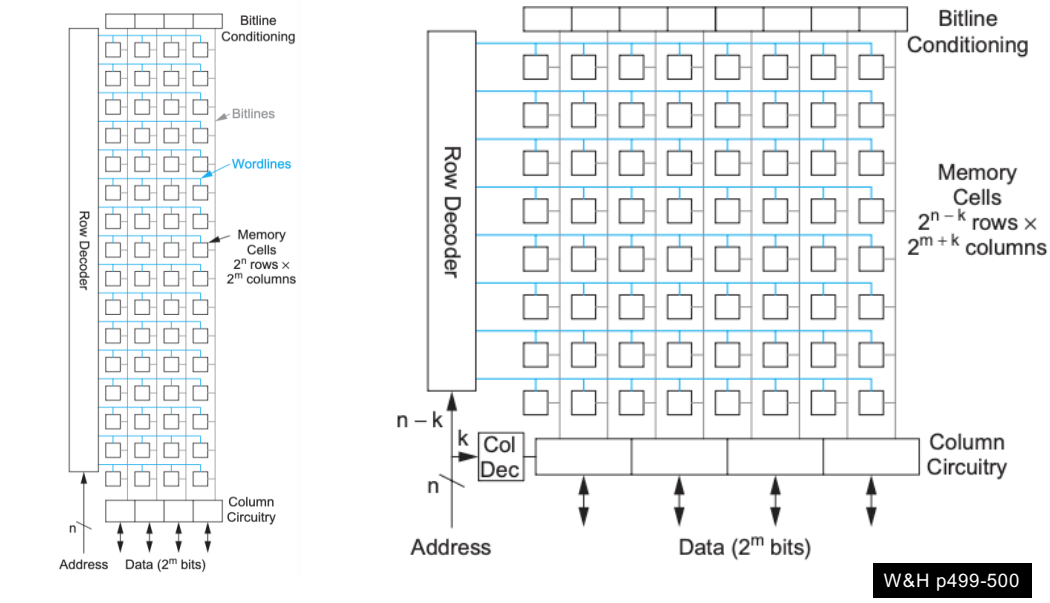
Categories of Memory Arrays



Memory arrays account for a large proportion of the silicon area on a chip, often in the form of register files, embedded SRAM and cache memory. The diagram here shows the hierarchy of memory from VLSI design point of view.

In this lecture, we will examine the internal circuit of some of some of these memory types. In particular, we will examine the circuits for SRAM, DRAM, flash RAM and content addressable memory (CAM).

Memory Array Architecture



W&H p499-500

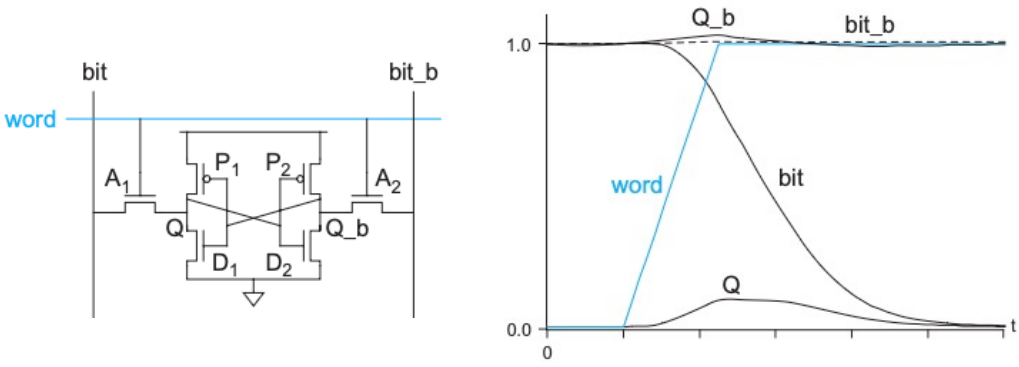
A n x m memory array contains 2^n words of 2^m bits each. Each bit is stored in a memory cell, which takes up most of the silicon area. Shown on the left is a small memory array containing 16 4-bit words (n = 4, m = 2). This show a simple design with with one row per word and one column per bit. The row decoder uses the address to activate one of the rows by asserting the wordline.

During a read operation, the cells on this wordline drive the bitlines, which are often conditioned to a known value in advance of the memory access. The column circuitry usually contain amplifiers or buffers to sense the data.

A typical memory array may have thousands or millions of words of 8–64 bits each, which would lead to a tall, skinny layout that is hard to fit in the chip floorplan and slow because of the long vertical wires. Therefore, the array is often folded into fewer rows of more columns. After folding, each row of the memory contains 2^k words, so the array is physically organized as 2^{n-k} rows of 2^{m+k} columns or bits.

The figure on the right shows a two-way fold (k= 1, 2^k = 2) with eight rows and eight columns. The column decoder controls a multiplexer in the column circuitry to select 2^m bits from the row as the data to access. Larger memories are generally built from multiple smaller subarrays so that the wordlines and bitlines remain reasonably short, fast, and low in power dissipation.

Read operation for SRAM cell



W&H p500

SRAMs are widely used for register files, scratchpad buffer, FIFOs and caches. They have the following attractive properties: 1) denser than flip-flops; 2) compatible with standard CMOS processes; 3) faster than DRAM; 4) easier to use than DRAM.

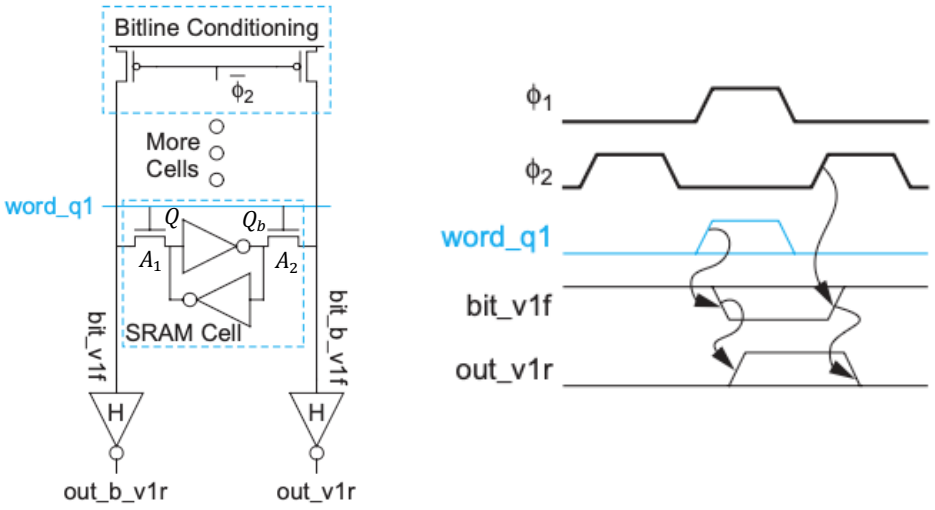
A common SRAM cell circuit contains a pair of weak cross-coupled inverters holding the state and a pair of access transistors to read or write the state. The positive feedback corrects disturbances caused by leakage or noise.

The cell is written by driving the desired value and its complement onto the bitlines, ***bit*** and ***bit_b***, then raising the wordline, ***word***. The new data overpowers the cross-coupled inverters. It is read by precharging the two bitlines high, then allowing them to float. When word is raised, ***bit*** or ***bit_b*** pulls down, indicating the data value.

The timing diagram on the right shows a SRAM cell being read. The bitlines are both initially floating high. Assume ***Q*** is initially 0 and thus ***Q_b*** is initially 1. ***Q_b*** and ***bit_b*** both should remain 1. When the wordline is raised, bit should be pulled down through driver and access transistors ***D1*** and ***A1***. At the same time ***bit*** is being pulled down, node ***Q*** tends to rise. ***Q*** is held low by ***D1***, but raised by current flowing in from ***A1***. Hence, the driver ***D1*** must be stronger than the access transistor ***A1***.

Note that ***Q*** momentarily rises due to the charge stored on the bit line being shared.

Column Read Operation



W&H p501

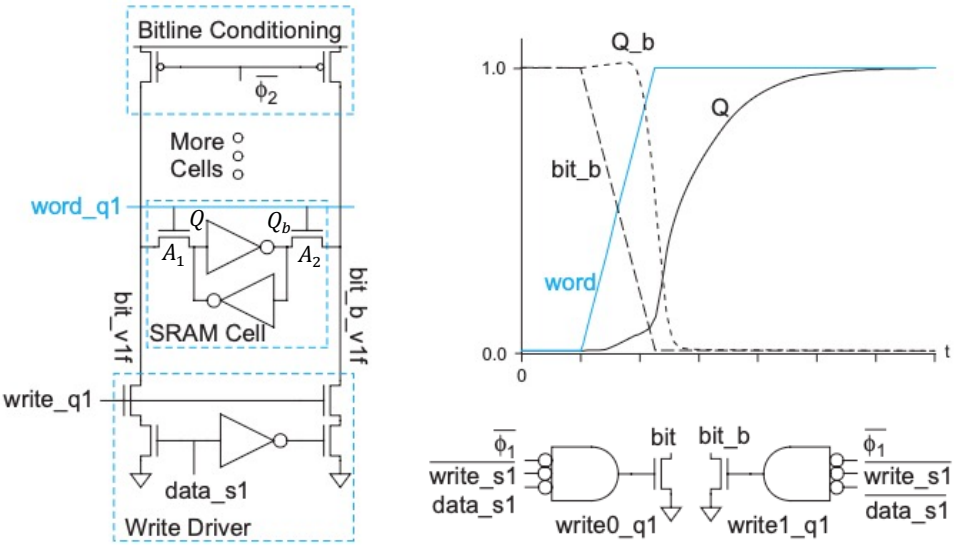
The circuit above shows the SRAM cell as part of a column of cells. Reading of the cell content is synchronous to a two-phase clock, which is shown to be non-overlapping (but could be simply single-phase with clock and clock_bar).

During phase 2, the bitlines are precharged high by the row of pull-up pMOS transistors. This is known as “conditioning”.

The wordline rises during phase 1 and the SRAM cell forces its stored value onto the two bit lines. The capacitance of the entire bitline must be discharged through the access transistors that connect the cross-coupled inverter outputs to the two bitlines.

The output can be sensed by a pair of inverters which are designed to have a high switching threshold (called a “HI-skew” inverter). By raising the switching threshold of the sense inverters, delay can be reduced at the expense of noise margin.

Write Operation



W&H p501

The timing diagram above shows the SRAM cell, which initially stores a 0, but is s being written into the cell with a 1.

bit is precharged high and left floating. ***bit_b*** is pulled low by a write driver. The cell is written by forcing ***Q_b*** low through A2. The pull-up pMOS opposes this operation; thus, it must be weaker than A2 so that the inverter output can be pulled low enough to flip the stored state. This constraint is called writability.

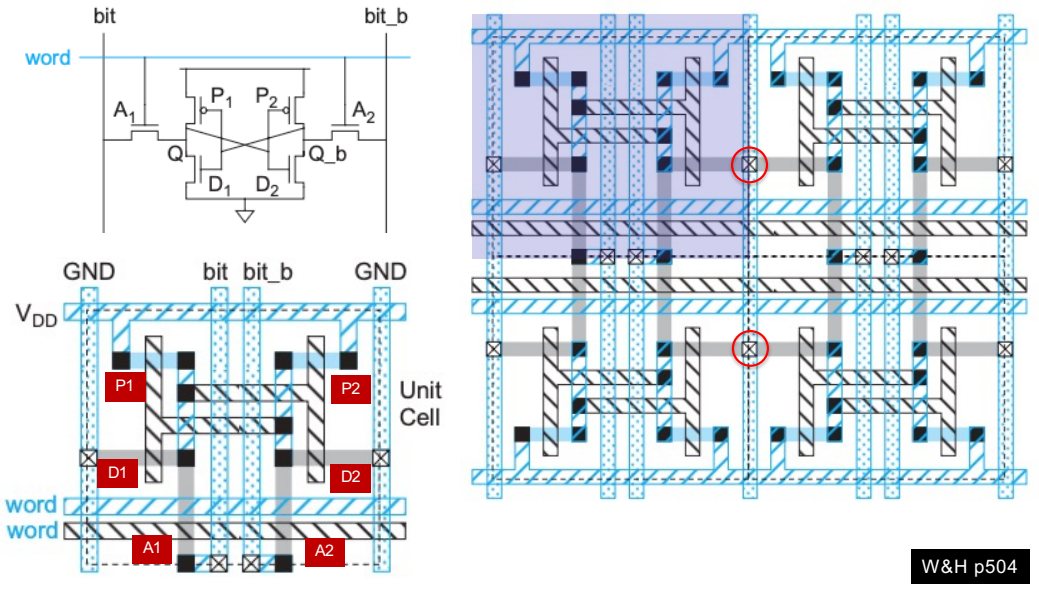
The circuit above shows the SRAM cell in a column of cells. During phase 2, the bitlines are precharged high.

Write drivers pull the bitline or its complement low during phase 1 to write the cell. The write drivers circuit on the bottom right shows a single transistor driven by the appropriate combination of signals to drive the bitlines during a write operation.

The series resistance of the write driver, bitline wire, and access transistor must be low enough to overpower the pMOS transistor in the SRAM cell.

The central challenges in SRAM design are minimizing its size and ensuring that the circuitry holding the state is weak enough to be overpowered during a write, yet strong enough not to be disturbed during a read. Therefore, SRAMs are usually provided by the silicon vendor as a highly optimized and thoroughly verified IP block. In our case, we rely on TSMC's 65nm SRAM compiler to produce the necessary SRAM module for our designs.

6T SRAM cell Layout (stick diagram)

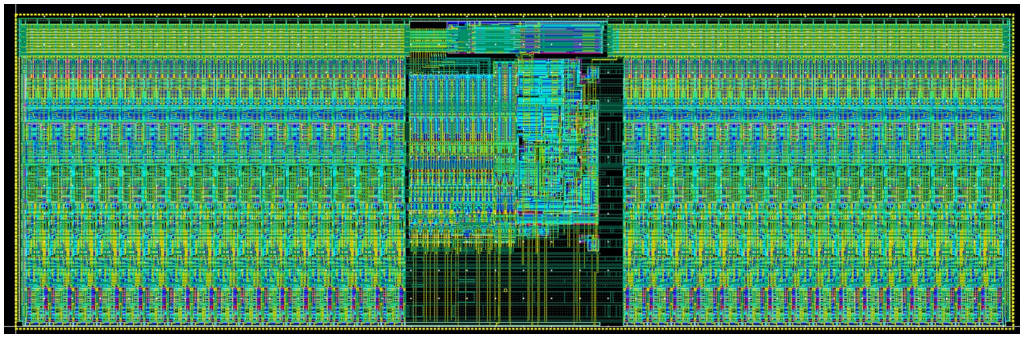


The above diagrams shows a stick diagram of a traditional 6T cell. The cell is designed to be mirrored and overlapped to share VDD and GND lines between adjacent cells along the cell boundary.

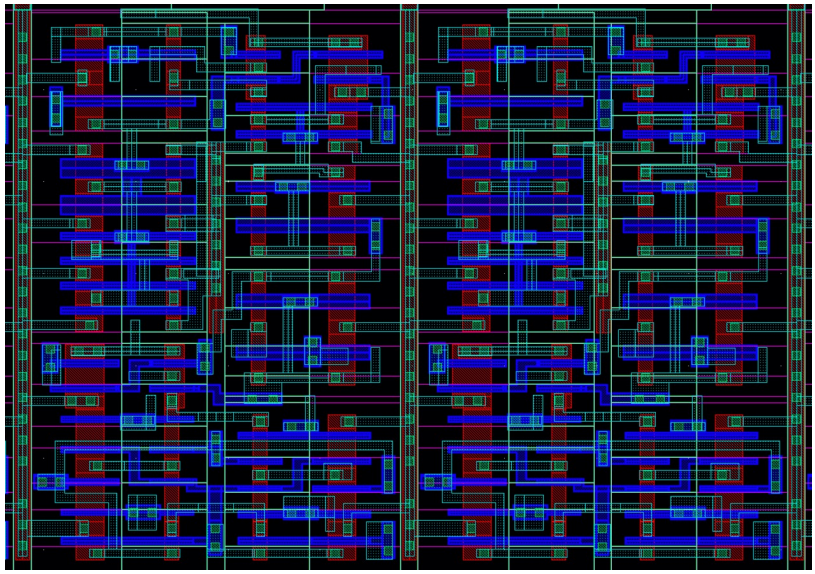
Note how a single diffusion contact to the bitline is shared between a pair of cells. This halves the diffusion capacitance, and hence reduces the delay discharging the bitline during a read VDD access. The wordline is run in both metal1 and polysilicon; the two layers must occasionally be strapped (e.g., every four or eight cells).

Since we rely on TSMC's SRAM compiler to produce for us the layout of SRAM blocks, it is not necessary for you to explore this layout in detail.

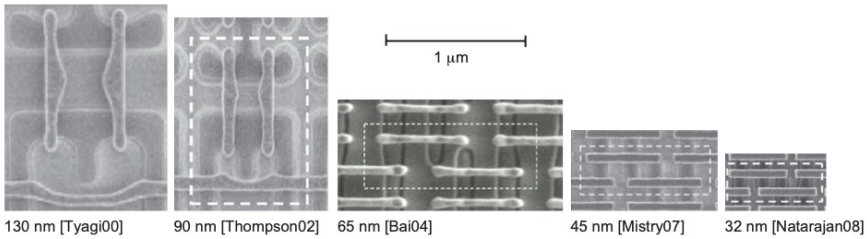
32 x 32 SRAM

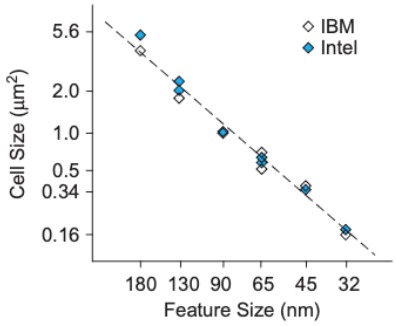


Zoom-in RAM cells



SRAM Scaling

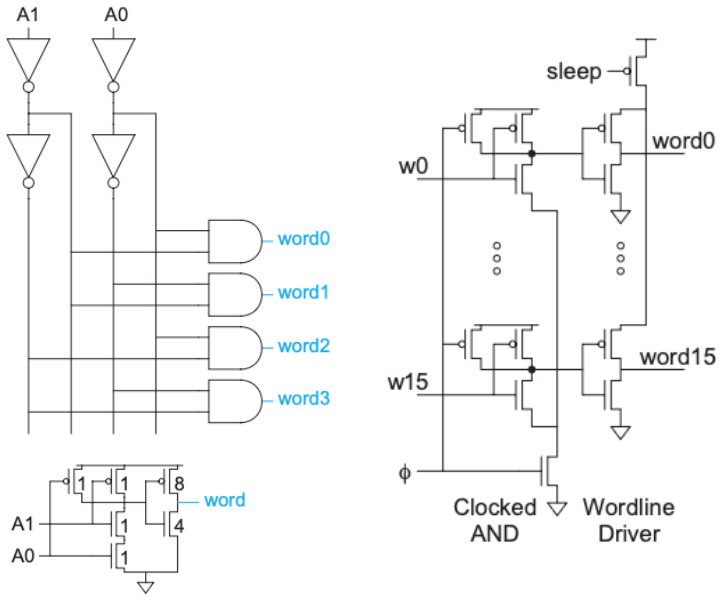




W&H p505

This diagram shows that SRAM scales very well with advancement in technology despite the growing challenges of lithography and variability. SRAM is so important that design rules are scrutinized and bent where possible to minimize cell area in commercial processes. The substrate and well contacts are shared among multiple cells to save area at the expense of regularity.

Row Decoder Circuits



W&H p506-507

Cicuit on the left:

The row circuitry consists of the decoder and word line drivers. The simplest decoder is a collection of AND gates using true and complementary versions of the address bits. The above design on the bottom left is a static NAND gate followed by an inverter. This structure is useful for up to 5–6 inputs or more if speed is not critical. The NAND transistors are usually made minimum size to reduce the load on the buffered address lines because there are 2^{n-k} transistors on each true and complementary address line in the row decoder.

Cicuit on the right:

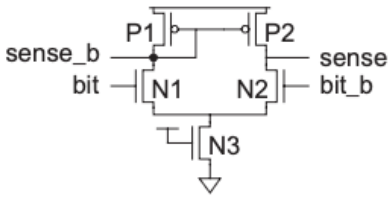
The wordline generally must be qualified with the clock for proper bitline timing. This is often performed with another AND gate after the decoder or with an extra clock input to the final stage of decoding as shown in the circuit on the right.

This circuit takes advantage of the 1-hot nature of decoder outputs to share the clocked nMOS transistor across multiple final 2-input AND gates, reducing wordline clock power.

The wordline driver inverters are large and contribute a significant amount of leakage current. At most one driver produces a ‘1’ output at a time. The circuit also includes a sleep transistor that cuts off leakage for the drivers in the 0 state when the array is inactive. The sleep transistor only needs to be wide enough to supply current to a single inverter.

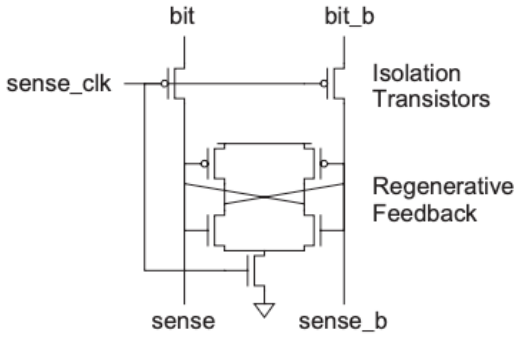
Sense Amplifier

Differntial Sense Amplifier



(a)

Clocked Sense Amplifier



(b)

W&H p511

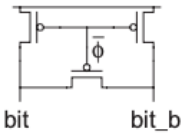
The column circuitry includes the sense amplifier that detects a bitline's state, the column decoder that decodes the column address, and the output multiplexer to select the addressed column to drive the output signals.

The differential sense amplifier (a) is based on an analogue differential pair and requires no clock. However, the circuit consumes a significant amount of DC power. It is also difficult to bias at low voltage to keep all the transistors in saturation.

Therefore a clocked sense amplifier shown on the right is commonly used. It consumes power only while activated, but requires a timing chain to activate at the proper time.

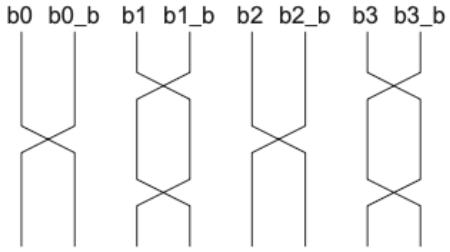
When the sense clock is low, the amplifier is inactive. When the sense amplifier rises, it effectively turns on the cross-coupled inverter pair, which pulls one output low and the other high through regenerative feedback. The isolation transistors speed up the response by disconnecting the outputs from the highly capacitive bitlines **during sensing**.

Differential Noise Reduction



(a)

Bitlines equalization



(b)

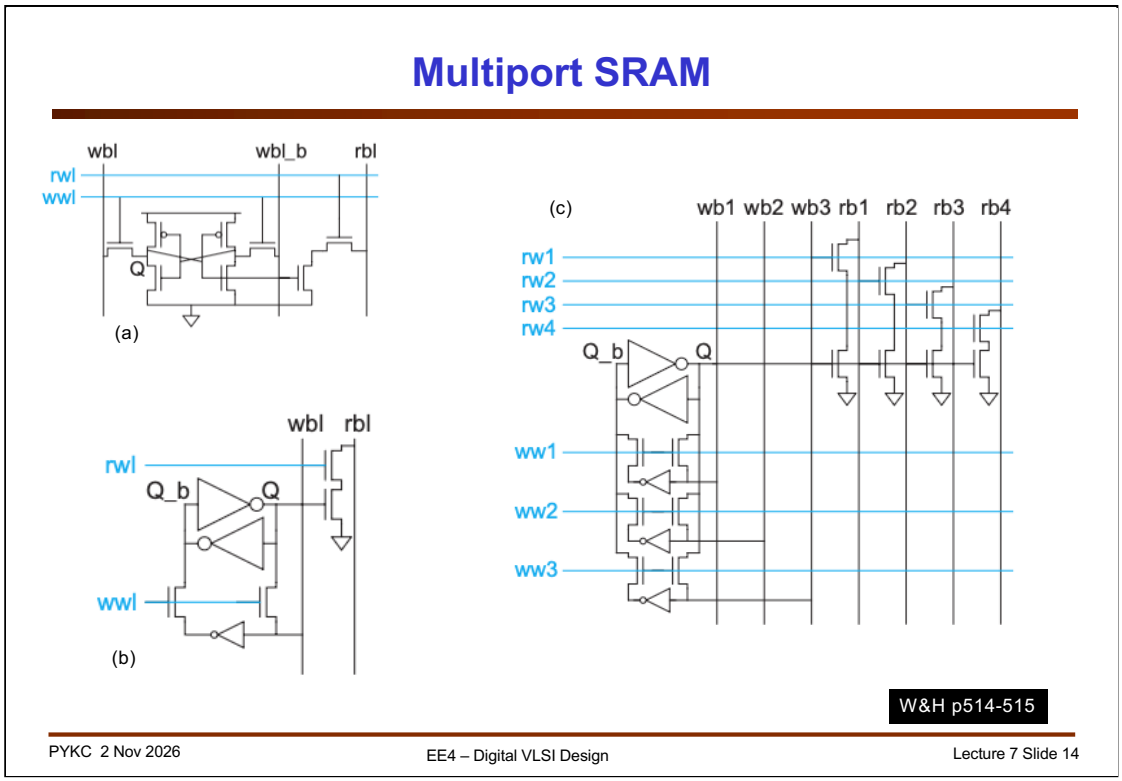
Bitlines twisting

W&H p513

Sense amplifiers are highly susceptible to **differential noise** on the bitlines because they detect small voltage differences (100mV to 300mV). If bitlines are not precharged long enough, residual voltages on the lines from the previous read may cause pattern-dependent failure.

An **equalizer transistor** (shown in (a) above) can be added to the bitline conditioning circuits to reduce the required precharge time by making sure that the bit and bit_b voltages are equal before sensing.

Another technique to minimise the impact of differential noise is to use twisted bitlines as shown in (b). This causes equal coupling onto both the bitline and its complement.



Register files are generally fast SRAMs with **multiple read and write ports**.

The top left diagram showed a conventional 8T dual-ported SRAM cell with one write port and one read port. Note that the read port is single-ended.

To reduce the size of multiport SRAM cell, one could use **single-ended sens**ing for both read and write ports, as shown in diagram (b) above.

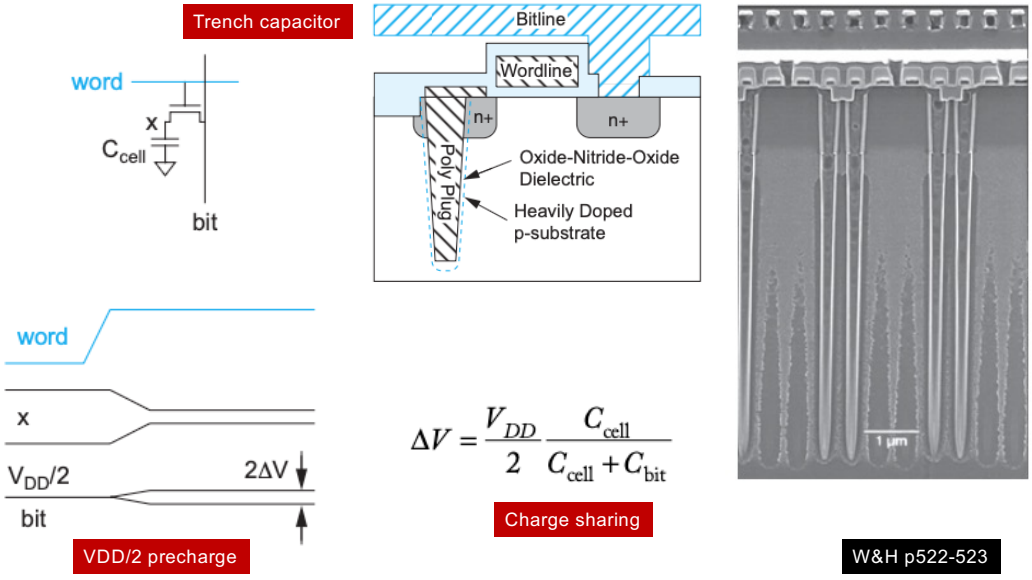
Each additional single-ended read port can be provided at the cost of a read wordline, a read bitline, and two read transistors. Differential read ports double the number of read bitlines and transistors.

Cells with **multiple write ports** simply attach the ports to the state node. External logic should ensure that two ports do not attempt to simultaneously write different values to the register. Each additional write port can be provided at the cost of a write wordline, true and complementary write bitlines, and two access transistors.

For cells with many ports, the area of the wires dwarfs the area of the transistors. To save space, the complementary write bitline can be eliminated by adding a transistor or inverter within the cell as shown in the right most diagram. This circuit has four read ports and three write ports.

The inverter approach requires one more transistor but improves the writability (i.e. reduce chance of a write error). Since In multiport memory, the silicon area is dominated by the extra interconnection, this extra transistor is not significant in area efficiency.

DRAM cell



Dynamic RAMs (DRAMs) store their contents as charge on a capacitor rather than in a pair of cross-coupled inverter. The basic cell is substantially smaller than SRAM, but the cell must be **periodically read and refreshed** so that its contents do not leak away.

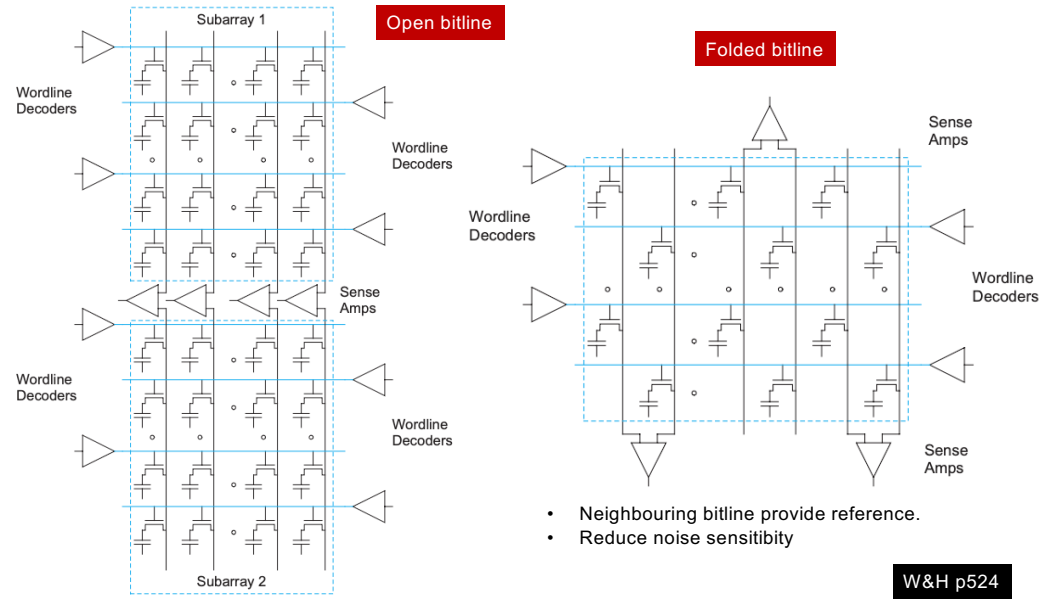
DRAMs are built in specialized processes optimized for dense capacitor structures and offer a factor of 10–20 greater density (bits/cm^2) than high-performance SRAM built in a standard logic process.

A 1-transistor (1T) dynamic RAM cell consists of a transistor and a capacitor, as shown above. Like SRAM, the cell is accessed by asserting the wordline to connect the capacitor to the bitline. On a read, the bitline is first **precharged to VDD/2**. When the wordline rises, **the capacitor shares its charge** with the bitline, causing a voltage change (ΔV) that can be sensed as shown in the timing diagram above. The read disturbs the cell contents at x, so the cell must be rewritten after each read. The change in bitline voltage ΔV is determined by the charge sharing equation above.

On a write, the bitline is driven high or low and the voltage is forced onto the capacitor. Some DRAMs drive the wordline to VDDP = VDD + V_t to avoid a degraded level when writing a '1' through the nMOS transistor.

To ensure that cell capacitance C_{cell} is sufficiently large, **trench capacitors** etched under the source of the transistor is used as shown above. The walls of the trench are lined with an oxide-nitride-oxide dielectric. The trench is then filled with a polysilicon conductor that serves as one terminal of the capacitor attached to the transistor drain, while the heavily doped substrate serves as the other terminal. This shows that DRAM process is NOT the same as a conventional process for logic and SRAMs.

DRAM array

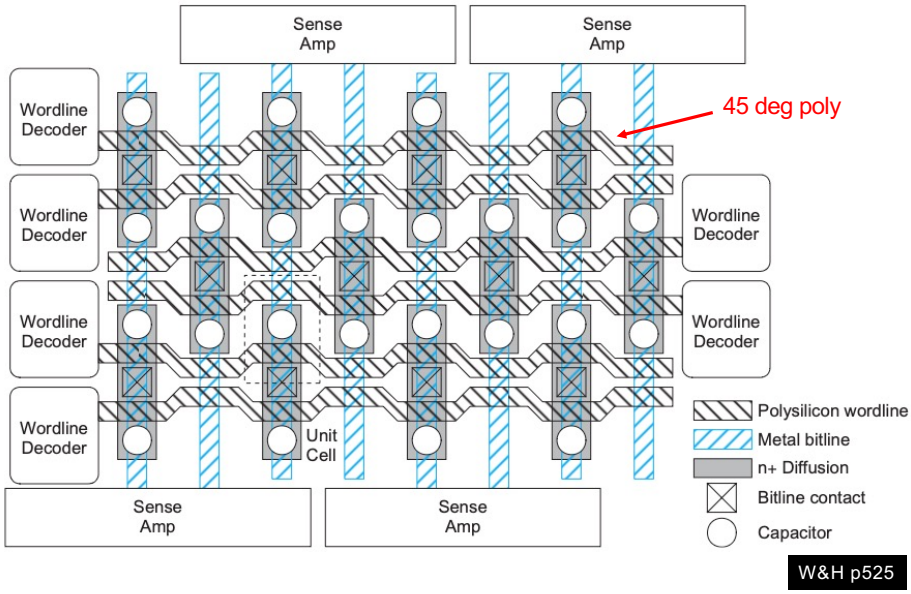


The structure of a DRAM array is very similar to those found in SRAM. It consists of wordlines and bitlines, wordline decoders and column decoders, bitline sense amplifiers as shown in the left diagram above.

The diagram on the right shows the **folded bitline architecture** where each bitline connects to only half as many cells. Adjacent bitlines are organized in pairs as inputs to the sense amplifiers.

When a wordline is asserted, one bitline will switch while its **neighbour serves as the reference**. Many noise sources will couple equally onto the two adjacent bitlines so they tend to appear as common mode noise that is rejected by the sense amplifier.

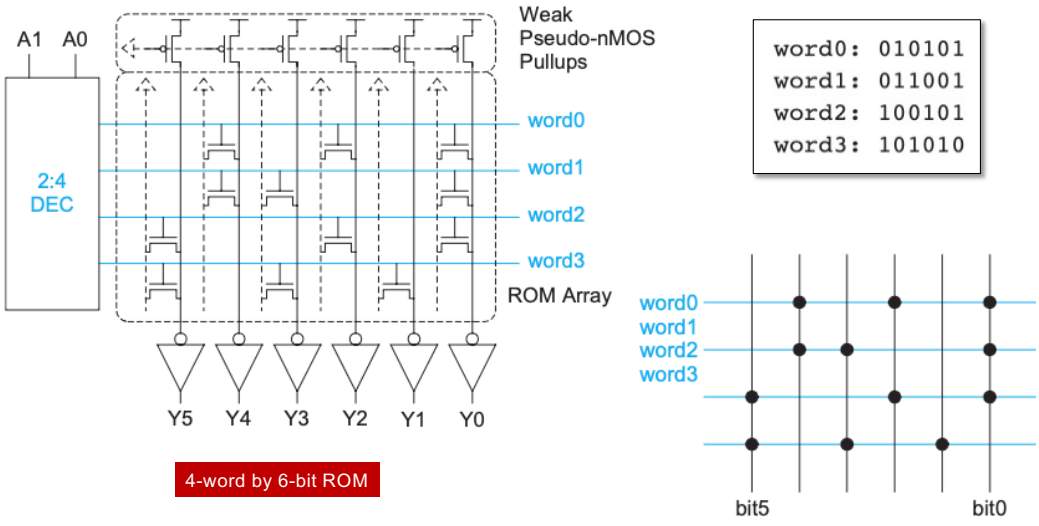
Folded Bitline Layout



This noise advantage comes at the expense of greater layout area. The layout above shows a clever layout for a 6 × 8 folded bitline subarray that is only **33% larger** than an open bitline layout.

DRAM processes push the design rules and use **diagonal polysilicon** to reduce area as shown in the layout above. Pairs of cells in the layout share a single bitline contact to minimize the bitline capacitance.

Pseudo nMOS ROM Circuit



4-word by 6-bit ROM

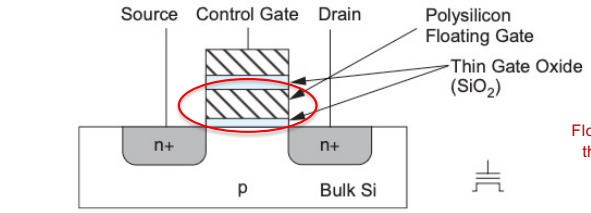
W&H p527

The above diagram shows a 4-word by 6-bit ROM using pseudo-nMOS pullups.

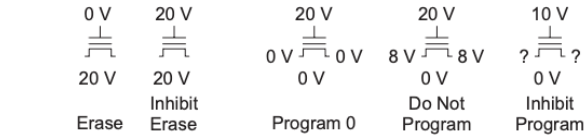
The contents of the ROM can be symbolically represented with a dot diagram in which dots indicate the presence of 1s. The dots correspond to nMOS pulldown transistors connected to the bitlines, but the outputs are inverted.

Mask-programmed ROMs can be configured by the presence or absence of a transistor. Omitting transistors has the advantage of reducing capacitance on the wordlines and power consumption. Mask-programmable ROMs are now outdated, replaced by EEPROM and **Flash memory technology**.

Flash Memory Cell



Floating gate traps electrons through Fowler-Nordheim tunnelling



Substrate raised to 20V – cells programmed 1

Gate raised to 20V - trap electrons raises V_t

W&H p531

Flash memory cell uses a second layer of polysilicon to form a floating gate between the control gate and the channel, as shown above.

The floating gate is a good conductor, but it is not attached to anything. Applying a high voltage to the control gate causes electrons to tunnel through the thin oxide onto the floating gate through the processes called **Fowler-Nordheim (FN) tunnelling**. Injecting the electrons induces a negative voltage on the floating gate, effectively increasing the threshold voltage of the transistor to the point that it is always OFF.

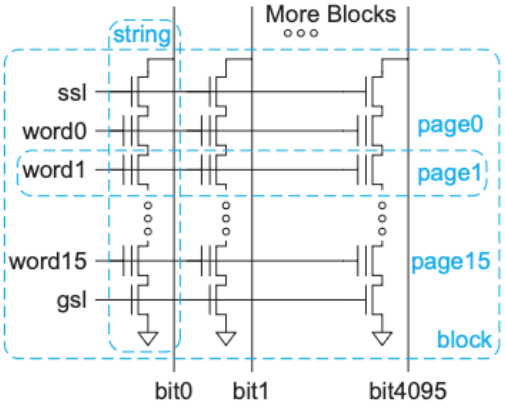
To **erase a block** of flash memory cells, the substrate is raised to 20 V while keeping all the control gates to GND. The high voltage across the gate oxide induces FN tunnelling, causing the electrons to flow from the floating gate to the substrate. At the end of the erase step, all the floating gate transistors have a negative V_t and thus represent 1.

Tunnelling is a slow process, so block erase takes on the order of a millisecond. The wordlines for other blocks on the chip are set to the same voltage as the substrate to inhibit erasing.

A cell is programmed to 0 by tunnelling electrons onto the floating gate. The programming cannot restore 1 values, so the block must be erased before any cell is reprogrammed. An entire page is programmed at once. To program a page, the bitlines are driven with the data values: 0 V for a logic 0 and 8 V for a logic 1. The substrate is held at ground. The wordline is set to 20 V for the page being programmed and 10 V for the other pages in the block.

An on-chip **charge pump circuit** is used to generate the high voltages.

Flash Memory Array



Each cell can store multibit value!



W&H p531

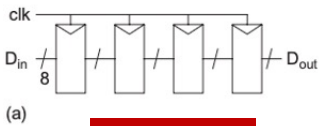
In NAND Flash memory shown in the diagram on the left, the floating gate transistors are connected in series to form strings, pages and blocks. Each string consists of 16 cells, a string select transistor, and a ground select transistor all connected in series and attached to the bitline.

The control gate of each cell is connected to a wordline. The array contains one column for each bit in a page. Each column contains one string per block. The number of cells in the string determines the number of pages per block.

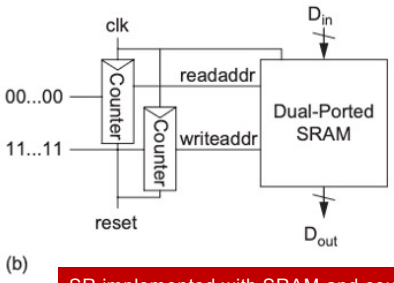
To achieve higher densities, **multilevel Flash cells store more than one bit** on a transistor by programming the threshold to one of several levels. The threshold can be sensed by adjusting the voltage on the selected wordline. The number of bits that can be stored depends on how accurately the threshold can be programmed and sensed.

The die photograph on the right is a 64 Gb NAND Flash chip from Toshiba using a 43 nm process with 3 metal layers. The chip uses a 16-level cell to store 4 bits per transistor. The memory is divided into two 32 Gb (4 GB) panes that can operate independently to double the throughput. Each pane has 64K columns. Hence, each page is 64 Kb (8 KB). Each string contains 64 series transistors. Thus, each block holds (64 transistors/string) × (4 bits/transistor)= 256 pages, or 2 MB of data. Each pane has 2K blocks. The chip operates at 3.3 V and has a programming bandwidth of 5.6 MB/s.

Shift Registers

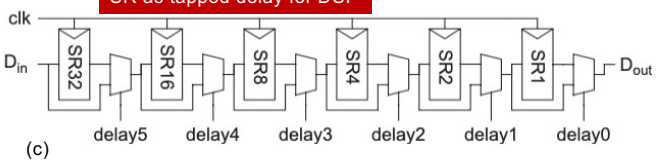


8-bit 4-stage SR



SR implemented with SRAM and counters

SR as tapped delay for DSP



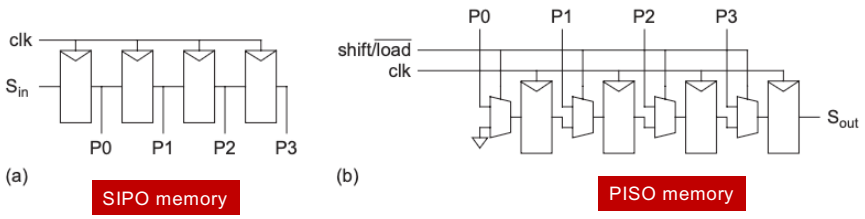
W&H p534

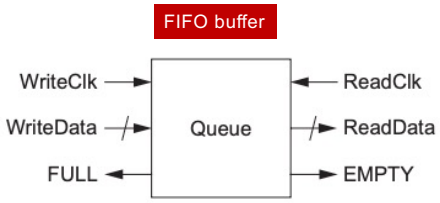
Shift register is commonly used in signal-processing applications to store and delay data. The diagram (a) above shows a simple 4-stage 8-bit shift register constructed from 32 flip-flops. As there is no logic between the registers, particular care must be taken that hold times are satisfied.

Flip-flops are expensive in area. Therefore, dense shift registers use dual-port RAMs instead as shown in (b). The RAM is configured as a circular buffer with a pair of counters specifying where the data is read and written.

The tapped delay line is a shift register variant that offers a variable number of stages of delay. (c) shows a 64-stage tapped delay line that could be used in a video processing system. Delay blocks are built from 32-, 16-, 8-, 4-, 2-, and 1-stage shift registers. Multiplexers allow the delay blocks be by-passed to allow programmable delay.

Delay Line and FIFO



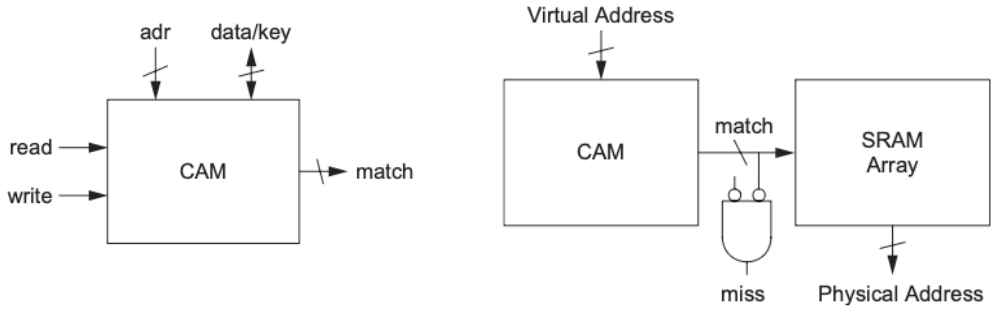


W&H p534

Another use of shift registers are to implement **Serial In Parallel Out** (SIPO) memory and **Parallel In Serial Out** (PISO) memory. These are also often useful in signal processing and communications systems.

First In First Out (FIFO) queues are commonly used to buffer data between two asynchronous streams. Like a shift register, the FIFO is organized as a circular buffer. On reset, the read and write pointers are both initialized to the first element and the FIFO is EMPTY. On a write, the write pointer advances to the next element. If it is about to catch the read pointer, the FIFO is FULL. On a read, the read pointer advances to the next element. If it catches the write pointer, the FIFO is EMPTY again. (See (c) above.). FIFO can be implemented with SRAM and counters.

Content Addressable Memory (CAM)



W&H p535

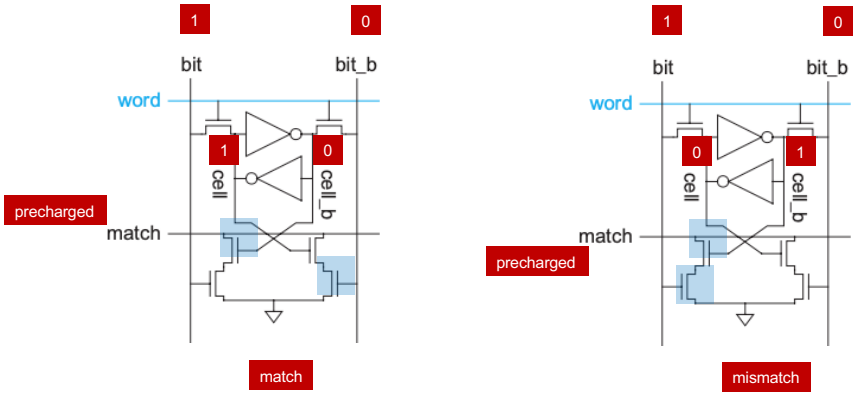
A **content-addressable memory (CAM)** function like a SRAM that can be read or written but also performs **matching operations**. Matching asserts a **matchline** output for each word of the CAM that contains a **specified key**.

CAM are used as part of cache memory in processors.

Another common application of CAMs is **translation lookaside buffers** (TLBs) in microprocessors supporting virtual memory. The virtual address is given as the key to the TLB CAM. If this address is in the CAM, the corresponding matchline is asserted. This matchline can serve as the wordline to access a RAM containing the associated physical address as shown above.

A NOR gate processing all of the matchlines generates a miss signal for the CAM.

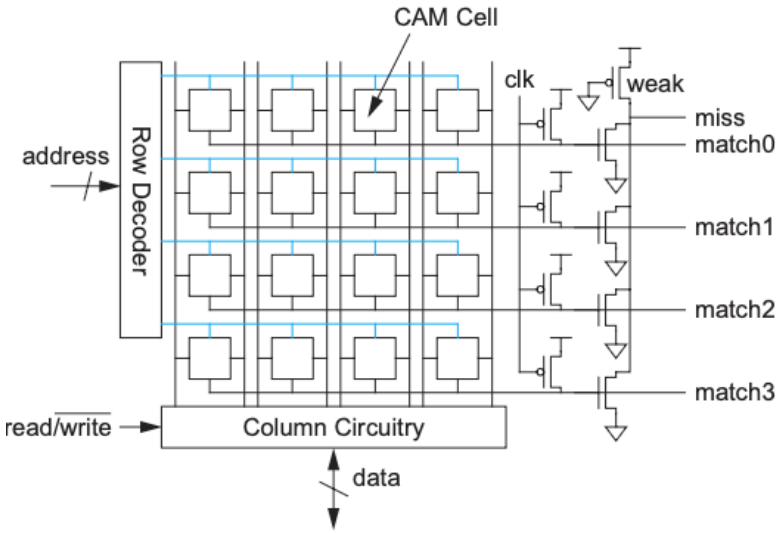
CAM Circuit (1)



W&H p535

The diagram above shows a 10T CAM cell consisting of a normal SRAM cell with **additional transistors to perform the match**. Multiple CAM cells in the same word are tied to the same matchline. The matchline is either precharged or pulled high as a distributed pseudo-nMOS gate. The key is placed on the bitlines. If the key and the value stored in the cell differ, the matchline will be pulled down. Only if all of the key bits match all of the bits stored in the word of memory will the matchline for that word remain high.

CAM Circuit (2)



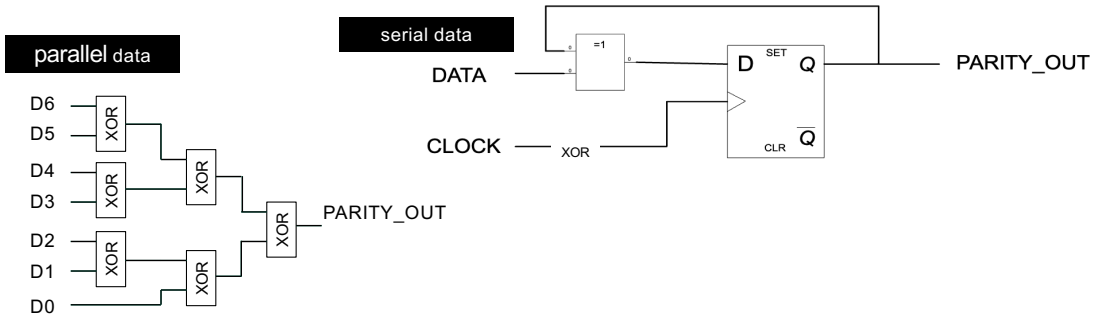
W&H p535

The figure above shows a complete 4 × 4 CAM array. Like an SRAM, it consists of an array of cells, a decoder, and column circuitry. However, each row also produces a dynamic matchline. The matchlines are precharged with the clocked pMOS transistors. The miss signal is produced with a distributed pseudo-nMOS NOR. Initially, all the matchlines are high. During CAM operation, the lines pull down, leaving at most one line asserted to indicate which row contains the key.

Error Detection in Memory

- Add extra data bit to ensure total number of 1's is EVEN – even parity
- Parity bit is the exclusive-OR of all other data bits
- If one of the received bit is wrong, then number of 1's will be odd (including parity bit) and error is detected

Data	Parity bit	number of 1's
1 1 0 1 1 1 0	1	6
0 1 0 1 1 0 1	0	4



Parity bit can be used to detect error in memory. The parity bit *p* is computed by XORing together all data bits. If there is an even number of '1's, the parity bit is 0. If the number of '1's is odd, then the parity is '1'. In other words, including the parity bit, the total number of '1's is always EVEN because *p* makes it even.

Consider the example shown above, 7'b1101110 has 5 '1's, so *p* = 1 to make 6 '1's. The lower number 7'b0101101 has 4 '1's, which is already even. So *p* = 0.

Computing *p* is straightforward. It only requires a tree of XOR gates as shown above.

For a serial data (a time sequence of '1's and '0's), the circuit here derives the parity output of that sequence. If PARITY_OUT is 0, the sequence has even parity (due to the XOR gate). If PARITY_OUT is 1, then the sequence has odd parity.

Stored or transmitted data must therefore include the parity bit. If there is a 1-bit error in the data or in the parity bit, there will be an odd number of '1's. Hence error is detected by simply summing (XORing) all the data + parity bit together.

Error Correction in Memory

- Simple parity is limiting in its ability to detect error
- It cannot CORRECT error
- Use multiple check bits, each giving parity for a DIFFERENT COMBINATION of data bits
- If a data bit is wrong, it will affect only some check bits – from which deduce WHICH bit is wrong:

	D0	D1	D2	D3	
P0:	x	x		x	P0 = D0 ⊕ D1 ⊕ D3
P1:	x		x	x	P1 = D0 ⊕ D2 ⊖ D3
P2:		x	x	x	P2 = D1 ⊕ D2 ⊕ D3
Code:	3	5	6	7	(0,1,2,4 not used)

- **Each data bit affects a UNIQUE combination of check bits**
- **Each data bit affects at least TWO check bits – this means an error in check bit is not confused with error in data bit**

Adding a parity bit detects a 1-bit error but cannot correct the error.

To correct error, we need to use multiple parity bits (also called check bits).

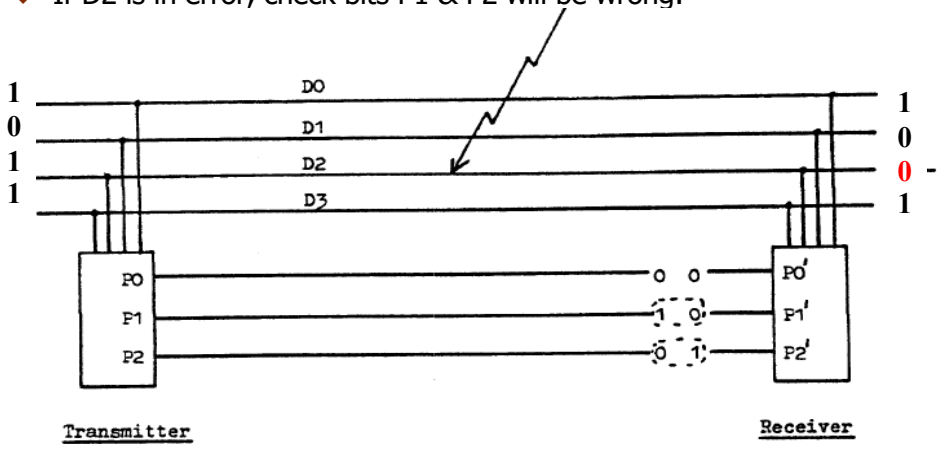
Consider 4-bit data D[3:0]. Let us introduce three parity bits P0, P1 and P2. P0 is the parity bit for D0, D1 and D3; P1 for D0, D2 and D3 etc. This is shown in the table above. That is, if D0 is wrong, P0 and P1 are wrong. If D3 is wrong, P0, P1 and P2 are all wrong. In that way, the patten of erroreous parity bits identifies the location of the 1-bit error.

Every data bit must affect at least TWO parity bits. Otherwise, an error with a parity bit will result in erroneous detection as a data bit error.

In the table above, the code for the parity bit affected by each data bit is shown in the row labelled “Code”. Note that codes 0, 1, 2, 4 cannot be used because only one or no parity bit will be affected by a data bit error – hence data bit error cannot be detected.

Example

- If D2 is in error, check bits P1 & P2 will be wrong:



Consider the example above. A 4-bit data 4'b1101 is transmitted (or stored) alongside its 3-bit parity 3'b010.

If D2 is flipped, the received data and parity are respectively 4'b1001 and 3'b010. But on the receiver end, the computed parity is 3'b001, which detects a mismatch in both P2 and P1. From the previous slide, the only single bit error resulting in these two check bits being different is if D2 is flipped.

The error detection circuit for D2 is therefore:

$$E2 = P2 \oplus P2' + P1 \oplus P1'$$

Corrected D2 is therefore simply:

$$Corrected_D2 = E2 \oplus Received_D2$$

Summary on Check Bits

- N check bits are sufficient for 2^N – N - 1 data bits
  - Each data bit must affect a different combination of check bits
  - There are 2^N possible combinations for N check bits
  - Of these, N affect only 1 bit and 1 affects no bits at all
- If you regard the combinations as binary numbers, then for 3 check bits, numbers 0, 1, 2 and 4 are ruled out, leaving 3, 5, 6, 7 as useful:

	D0	D1	D2	D3	
P0:	x	x		x	P0 = D0 ⊕ D1 ⊕ D3
P1:	x		x	x	P1 = D0 ⊕ D2 ⊖ D3
P2:		x	x	x	P2 = D1 ⊕ D2 ⊕ D3
Code:	3	5	6	7	(0,1,2,4 not used)

To summarize, N check bits are good enough to cover $2^N - N - 1$ data bits. This is because there are N cases where only 1 check bit is available, which is insufficient to detect if the 1-bit error is with data or parity. The -1 is for the case where the code is 0.

Using the binary number to indicate the association between check bits and data, then the code 0, 1, 2, 4 are not usable, leaving us with 3, 5, 6 and 7 for 4-bit data ECC.

ECC for 8-bit data

- Detect and correct any single bit error
- Can add an extra check bit giving overall parity to both data & check bits
  - will detect double bit errors but not correct
  - For double errors, overall parity bit will be OK, but check bits is wrong

	D0	D1	D2	D3	D4	D5	D6	D7	
P0:	x	x		x	x		x		
P1:	x		x	x		x	x		
P2:		x	x	x				x	
P3:					x	x	x	x	
Code:	3	5	6	7	9	10	11	12	(0,1,2,4,8 not used, 13,14,15 spare)

Here is the check bit assignment for 8-bit data with four check bits. For 8-bit data, 4 check bits are required.